

CUT YOUR LOSSES IN LARGE-VOCABULARY LANGUAGE MODELS

Разбор статьи

Бийелич М.Ю.

Проблема

По мере развития больших языковых моделей, растут и размеры их словарей. Это в свою очередь приводит к увеличению потребляемой памяти **на этапе обучения** на матрицу с логитами для расчёта лосса.

Особенно проблема критична для небольших моделей (1-8B), у которых память под матрицу логитов может занимать до 90% от всей потребляемой памяти.

model	V-size	E (N*D)	C (D*V)	Logits (V*N)
Gemma 2B	256 000	36 Mb	1.1 Gb	7.8 Gb
Llama 8B	128 256	64 Mb	1.0 Gb	3.9 Gb
Phi 3.5 mini	32 064	48 Mb	188 Mb	1.0 Gb

* E - матрица входных в классификационную голову эмбеддингов для N=8192 позиций текстовой последовательности.
C - матрица весов классификационной головы (проецирует скрытое состояние каждого токена текстовой последовательности в пространство словаря) - Logits
** E и C в bf16, логиты в fp32
*** Проблема с памятью проявляется только на обучении, где процесс предсказаний происходит в режиме Teacher Forcing.

Главная идея

Для расчёта loss'а нам необязательно материализовывать всю матрицу логитов $V \times N$.

Нам достаточно знать по 2 числа на каждую позицию текстовой последовательности.

x_i - правильный токен словаря V для позиции i текстовой последовательности, ему соответствует логит $A[x_i, i]$.
 LSE_i - log-sum-exp - операция, применяемая ко всему «столбцу» токенов словаря для позиции i - логарифма знаменателя софтмакса.

Т.е. в итоге для позиции i нам достаточно знать логит правильного токена и логарифм суммы экспонент в степени логитов всех токенов словаря вместо $|V|$ чисел.

А промежуточная матрица логитов нужна только для подсчёта LSE_i и $A[x_i, i]$ и её можно не хранить.

$$p(x_i) = \frac{e^{A[x_i, i]}}{\sum_k e^{A[k, i]}}$$

$$Loss_i = -\ln(p(x_i)) = -(A[x_i, i] - \ln Z_i) = \ln Z_i - A[x_i, i]$$

$$\ln Z_i = \ln\left(\sum_k e^{A[k, i]}\right) \implies L_i = LSE_i - A[x_i, i]$$

Пример

Допустим, мы сейчас обрабатываем текстовую последовательность «Кот сидит на ковре».

В режиме обучения модель по префиксу предсказывает следующее слово: после «кот» -> «сидит», после «кот сидит» -> «на», после «кот сидит на» -> «ковре».

Перемножим. Каждая клетка — логит «слово * позиция».

Рамкой выделен правильный ответ для каждого столбца.

Сама матрица здесь только для наглядности:

ССЕ её никогда целиком не строит.

Из каждого столбца нам нужны ровно две вещи:

число в рамке и LSE всего столбца.

$C^T (|V| \times D)$ — классификатор

	d_1	d_2		E_1 «КОТ»	E_2 «КОТ СИДИТ»	E_3 «КОТ СИДИТ НА»		
кот	-1	0	$\times$	d_1	1, 0	0, 0	0, 5	$=$
сидит	2	0		d_2	0, 0	1, 0	1, 0	
на	0	2						
ковре	1	1						
пёс	1	-1						
.	0	-1						

$A = C^T E (|V| \times N)$ — ЛОГИТЫ

	E_1 → сидит	E_2 → на	E_3 → ковре
кот	-1, 0	0, 0	-0, 5
сидит	2, 0	0, 0	1, 0
на	0, 0	2, 0	2, 0
ковре	1, 0	1, 0	1, 5
пёс	1, 0	-1, 0	-0, 5
.	0, 0	-1, 0	-1, 0

Цвет — величина логита. Рамка — правильный следующий токен x_j .

Пример

То есть CCE считает эти два столбца таблицы (логит и LSE) двумя отдельными ядрами, не записывая матрицу логитов (A) размером N×V в память.

$A = C^T E \ (|V| \times N)$ — логиты

	E ₁ → сидит	E ₂ → на	E ₃ → ковре
кот	-1,0	0,0	-0,5
сидит	2,0	0,0	1,0
на	0,0	2,0	2,0
ковре	1,0	1,0	1,5
пёс	1,0	-1,0	-0,5
.	0,0	-1,0	-1,0

Цвет — величина логита. Рамка — правильный следующий токен x_i .

Позиция	Правильно	Логит $A_{x,i}$	LSE _i	Loss = LSE – логит	$p = e^{-loss}$
«КОТ»	сидит	2,00	2.721	0.721	0.486
«КОТ сидит»	на	2,00	2.553	0.553	0.575
«КОТ сидит на»	ковре	1,50	2.783	1.283	0.277

Вычисления логитов

Наивно есть два варианта: посчитать всю матрицу логитов $A=C^T \cdot E$ и взять из неё нужные клетки (память $N \cdot |V|$), или сначала собрать матрицу нужных столбцов C_x (память $N \cdot D$), а затем сделать умножения на матрицу E .

CCE: для токена i загружает в SRAM столбец E_i и столбец C_{x_i} по индексу, считает скалярное произведение и пишет в HBM одно число.

В примере остальные 5 строк словаря для каждого токена не трогаются вообще.

На самом деле в реальном ядре это делается блоками: N_B токенов за раз, и по D тоже кусками D_B , аккумулируя сумму.
Сути это не меняет. Память сверх результата: 0.

Токен i	x_i	C_{x_i} (из SRAM)	E_i (из SRAM)	$C_{x_i} * E_i$	-> в HBM
1	сидит (#1)	(2, 0)	(1,0, 0,0)	$2 \cdot 1,0 + 0 \cdot 0,0$	2,0
2	на (#2)	(0, 2)	(0,0, 1,0)	$0 \cdot 0,0 + 2 \cdot 1,0$	2,0
3	ковре (#3)	(1, 1)	(0,5, 1,0)	$1 \cdot 0,5 + 1 \cdot 1,0$	1,5

Вычисления LSE

Со вторым слагаемым сложнее: LSE зависит от всех $|V|$ логитов столбца.
Решение: резать словарь на блоки по V_B токенов, для каждого блока считать свой маленький LSE и вливать его в общий.

Работает, потому что LSE ассоциативен:

$$\begin{aligned} LSE(a \cup b) &= \log(e^{LSE(a)} + e^{LSE(b)}) = \\ &= \max(LSE(a), LSE(b)) + \log(1 + e^{-|LSE(a) - LSE(b)|}) \end{aligned}$$

НВМ

глобальная память

Е, С (уже есть) + вектор LSE длины N. Матрицы логитов здесь нет.

LSE₁ = -∞

LSE₂ = -∞

LSE₃ = -∞

SRAM

пусто

LSE инициализирован значением -∞ для каждого токена.

Для сравнения: вся A. Серые клетки уже выброшены или ещё не посчитаны

	E ₁	E ₂	E ₃
кот	-1,0	0,0	-0,5
сидит	2,0	0,0	1,0
на	0,0	2,0	2,0
ковре	1,0	1,0	1,5
пёс	1,0	-1,0	-0,5
.	0,0	-1,0	-1,0

Вычисления LSE

Со вторым слагаемым сложнее: LSE зависит от всех $|V|$ логитов столбца.
Решение: резать словарь на блоки по V_B токенов, для каждого блока считать свой маленький LSE и вливать его в общий.

Работает, потому что LSE ассоциативен:

$$\begin{aligned} LSE(a \cup b) &= \log(e^{LSE(a)} + e^{LSE(b)}) = \\ &= \max(LSE(a), LSE(b)) + \log(1 + e^{-|LSE(a) - LSE(b)|}) \end{aligned}$$

HBM глобальная память

Е, С (уже есть) + вектор LSE длины N. Матрицы логитов здесь нет.

LSE₁ = 2,049

LSE₂ = 0,693

LSE₃ = 1,201

SRAM блок словаря 1: «кот», «сидит»

A_{nv} = C_v^T E_n, размер V_B×N = 2×3. После свёртки будет выброшен.

	E ₁	E ₂	E ₃
кот	-1,0	0,0	-0,5
сидит	2,0	0,0	1,0

n=1: LSE_блока = $\log(e^{-1,0} + e^{2,0}) = 2,049$; LSE = $\text{logaddexp}(-\infty, 2,049) = 2,049$
n=2: LSE_блока = $\log(e^{0,0} + e^{0,0}) = 0,693$; LSE = $\text{logaddexp}(-\infty, 0,693) = 0,693$
n=3: LSE_блока = $\log(e^{-0,5} + e^{1,0}) = 1,201$; LSE = $\text{logaddexp}(-\infty, 1,201) = 1,201$

Для сравнения: вся A. Серые клетки уже выброшены или ещё не посчитаны

	E ₁	E ₂	E ₃
кот	-1,0	0,0	-0,5
сидит	2,0	0,0	1,0
на	0,0	2,0	2,0
ковре	1,0	1,0	1,5
пёс	1,0	-1,0	-0,5
.	0,0	-1,0	-1,0

Вычисления LSE

Со вторым слагаемым сложнее: LSE зависит от всех $|V|$ логитов столбца.
Решение: резать словарь на блоки по V_B токенов, для каждого блока считать свой маленький LSE и вливать его в общий.

Работает, потому что LSE ассоциативен:

$$LSE(a \cup b) = \log(e^{LSE(a)} + e^{LSE(b)}) = \\ = \max(LSE(a), LSE(b)) + \log(1 + e^{-|LSE(a) - LSE(b)|})$$

НВМ глобальная память

Е, С (уже есть) + вектор LSE длины N. Матрицы логитов здесь нет.

LSE₁ = 2,440

LSE₂ = 2,494

LSE₃ = 2,721

SRAM блок словаря 2: «на», «ковре»

A_nv = C_v^T E_n, размер V_B×N = 2×3. После свёртки будет выброшен.

	E ₁	E ₂	E ₃
на	0,0	2,0	2,0
ковре	1,0	1,0	1,5

n=1: LSE_блока = log(e^{0,0} + e^{1,0}) = 1,313;

LSE = logaddexp(2,049, 1,313) = 2,440

n=2: LSE_блока = log(e^{2,0} + e^{1,0}) = 2,313;

LSE = logaddexp(0,693, 2,313) = 2,494

n=3: LSE_блока = log(e^{2,0} + e^{1,5}) = 2,474;

LSE = logaddexp(1,201, 2,474) = 2,721

Для сравнения: вся А. Серые клетки уже выброшены или ещё не посчитаны

	E ₁	E ₂	E ₃
кот	-1,0	0,0	-0,5
сидит	2,0	0,0	1,0
на	0,0	2,0	2,0
ковре	1,0	1,0	1,5
пёс	1,0	-1,0	-0,5
.	0,0	-1,0	-1,0

Вычисления LSE

Со вторым слагаемым сложнее: LSE зависит от всех $|V|$ логитов столбца.
Решение: резать словарь на блоки по V_B токенов, для каждого блока считать свой маленький LSE и вливать его в общий.

Работает, потому что LSE ассоциативен:

$$LSE(a \cup b) = \log(e^{LSE(a)} + e^{LSE(b)}) = \max(LSE(a), LSE(b)) + \log(1 + e^{-|LSE(a) - LSE(b)|})$$

HBM глобальная память

Е, С (уже есть) + вектор LSE длины N. Матрицы логитов здесь нет.

LSE₁ = 2,721

LSE₂ = 2,553

LSE₃ = 2,783

SRAM блок словаря 3: «пёс», «.»

A_nv = C_v^T E_n, размер V_B×N = 2×3. После свёртки будет выброшен.

	E ₁	E ₂	E ₃
пёс	1,0	-1,0	-0,5
.	0,0	-1,0	-1,0

n=1: LSE_блока = $\log(e^{1,0} + e^{0,0}) = 1,313$; LSE = $\text{logaddexp}(2,440, 1,313) = 2,721$
n=2: LSE_блока = $\log(e^{-1,0} + e^{-1,0}) = -0,307$; LSE = $\text{logaddexp}(2,494, -0,307) = 2,55$
n=3: LSE_блока = $\log(e^{-0,5} + e^{-1,0}) = -0,026$; LSE = $\text{logaddexp}(2,721, -0,026) = 2,78$

Готово: совпадает с прямым подсчётом по всему столбцу.

Для сравнения: вся A. Серые клетки уже выброшены или ещё не посчитаны

	E ₁	E ₂	E ₃
кот	-1,0	0,0	-0,5
сидит	2,0	0,0	1,0
на	0,0	2,0	2,0
ковре	1,0	1,0	1,5
пёс	1,0	-1,0	-0,5
.	0,0	-1,0	-1,0

Градиент без матрицы логитов

Так как мы не сохраняли в память матрицу логитов, на обратном проходе, чтобы корректно вычислить текущие от разных «слов» словаря градиенты, необходимо будет пересчитать их логиты A_{ij} .

Ключевой трюк: знаменатель softmax - LSE уже посчитан. Его сохранили в прямом проходе (всего N чисел), поэтому любой блок S считается независимо, без повторной синхронизации по словарю.

В CCE также делается оптимизация вычислений - если все градиенты текущего блока G_{nv} меньше некоторого малого ϵ , то последующие 2 дорогих матричных умножения градиентов на C и E и блокирующие сложения не производятся.

$$G = \partial L / \partial A = S - Y$$

$$S = \text{softmax}(A) = \exp(A - LSE)$$

$$Y = \text{onehot}(x)$$

$$\nabla E = C \cdot G(D \times N)$$

$$\nabla C = E \cdot G^T(D \times |V|)$$

```
for each pair (блок токенов n, блок словаря v):
    A_nv = C_v.T @ E_n                # пересчёт в SRAM
    G_nv = exp(A_nv - LSE_n) - onehot  # LSE_n из прямого прохода
    if all(|G_nv| < ε): skip           # ← фильтрация
    ∇E_n += C_v @ G_nv                 # atomic add в HBM
    ∇C_v += E_n @ G_nv.T               # atomic add в HBM
```

Фильтрация по градиентам

Почему мы можем делать фильтрации по градиенту и не делать вычислений для тех блоков, где все градиенты меньше ϵ ?

Потому что в bf16 на мантиссу 7 бит. При сложении двух чисел меньшее сдвигают к порядку большего, и младшие биты теряются. Если одно число меньше другого примерно в 2^8 раз, оно исчезает целиком, так как его значащий бит вылезает за пределы битности данного типа.

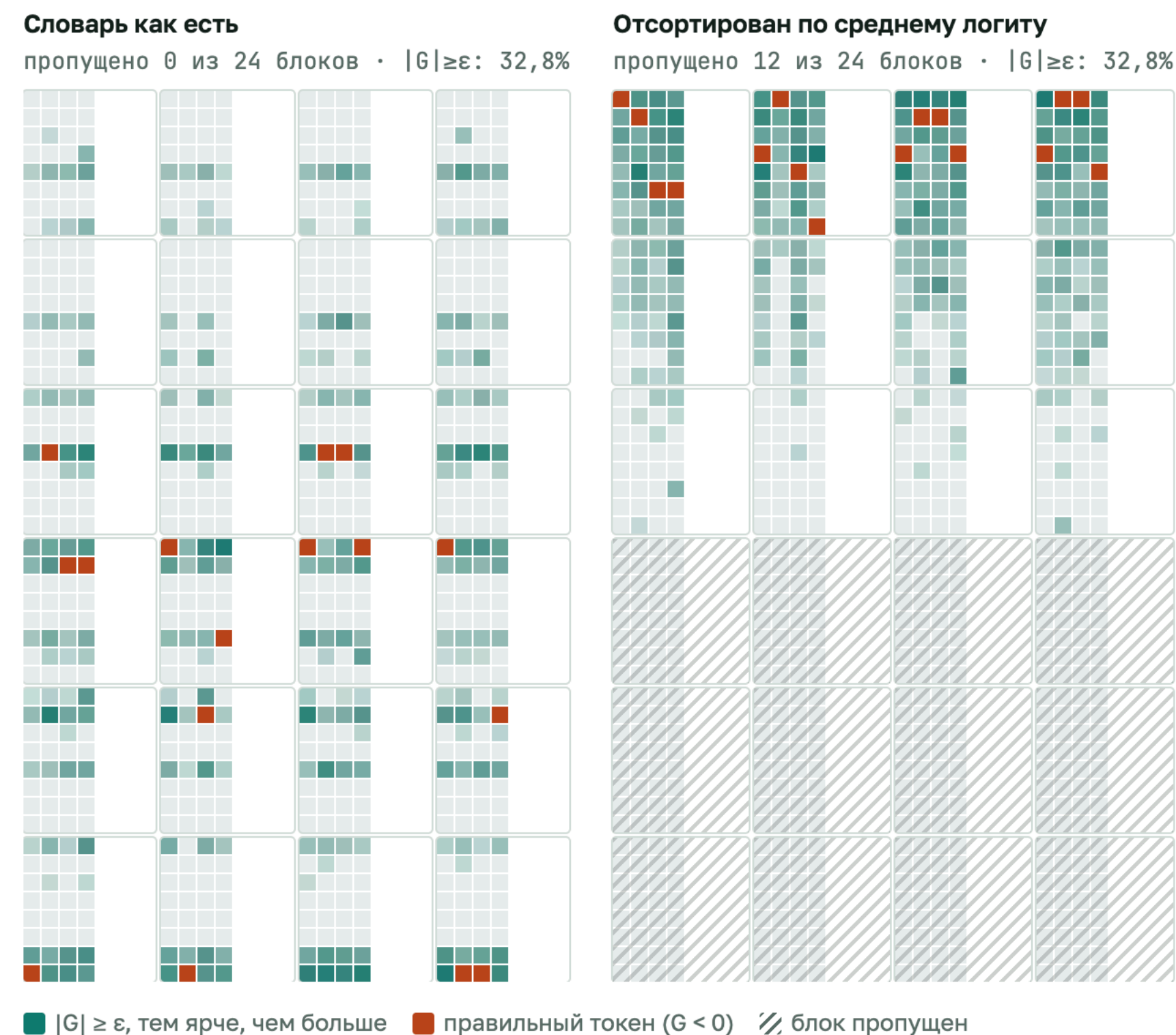
Помимо прочего в статье предлагается в процессе прямого прохода считать также среднее значение логитов для слова из словаря, а перед обратным проходом сортировать словарь по этим значениям, чтобы «группировать» в блоки в соответствии с их вероятностью «занулиться».

При этом вычисления градиентов на обратном проходе происходят всё равно честно, мы в любом случае сравниваем G с ϵ и если хотя бы один градиент из блока не меньше, то делаем вычисления.

$b = 1$ 0 01111111 00000000

$a = 2^{-8}$ 0 01110111 00000000

$b + a$ 0 01111111 00000000 = 1



Результаты

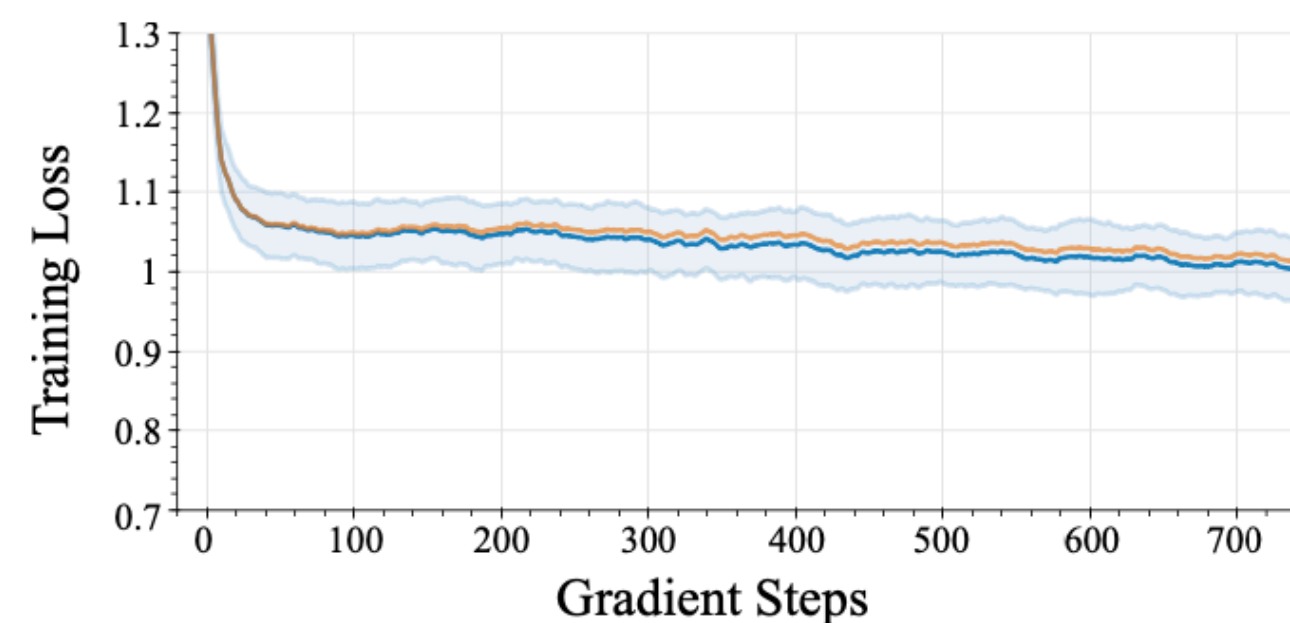
В таблице представлены замеры пикового потребления памяти и скорости работы на Gemma 2 2B на A100 80 GB, $|V| = 256\,000$, $D = 2304$ с 8192 токенами.



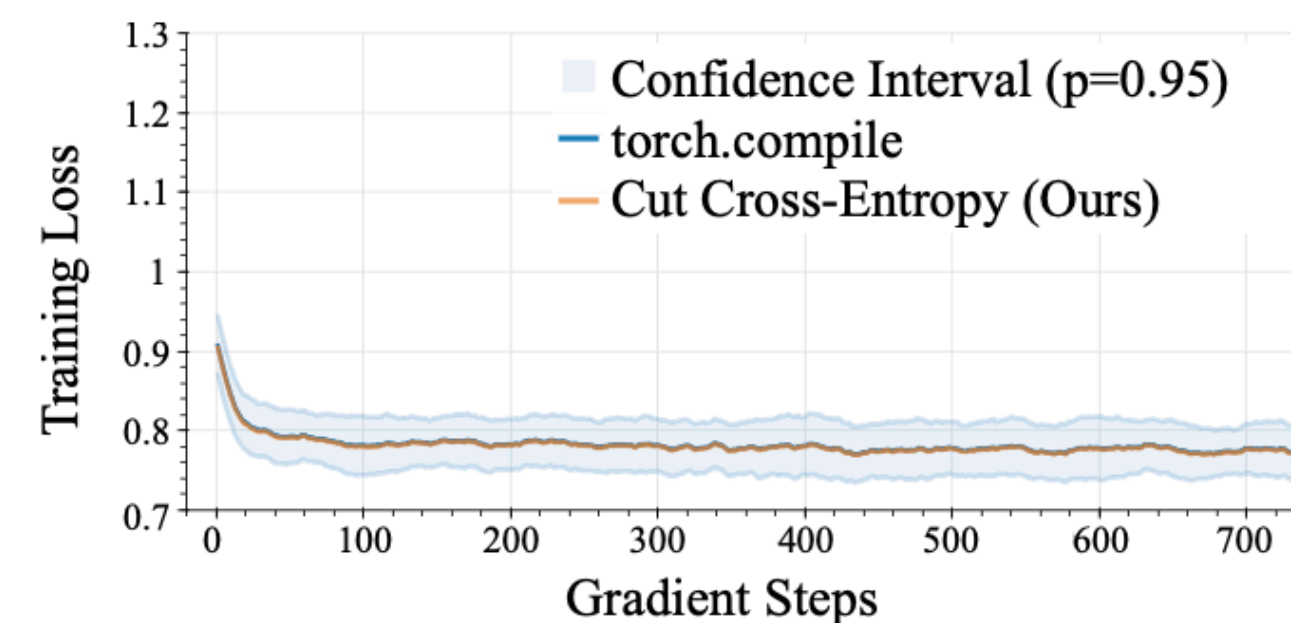
Результаты

На fine-tune кривые лосса для CCE неотличимы от обычного подхода.

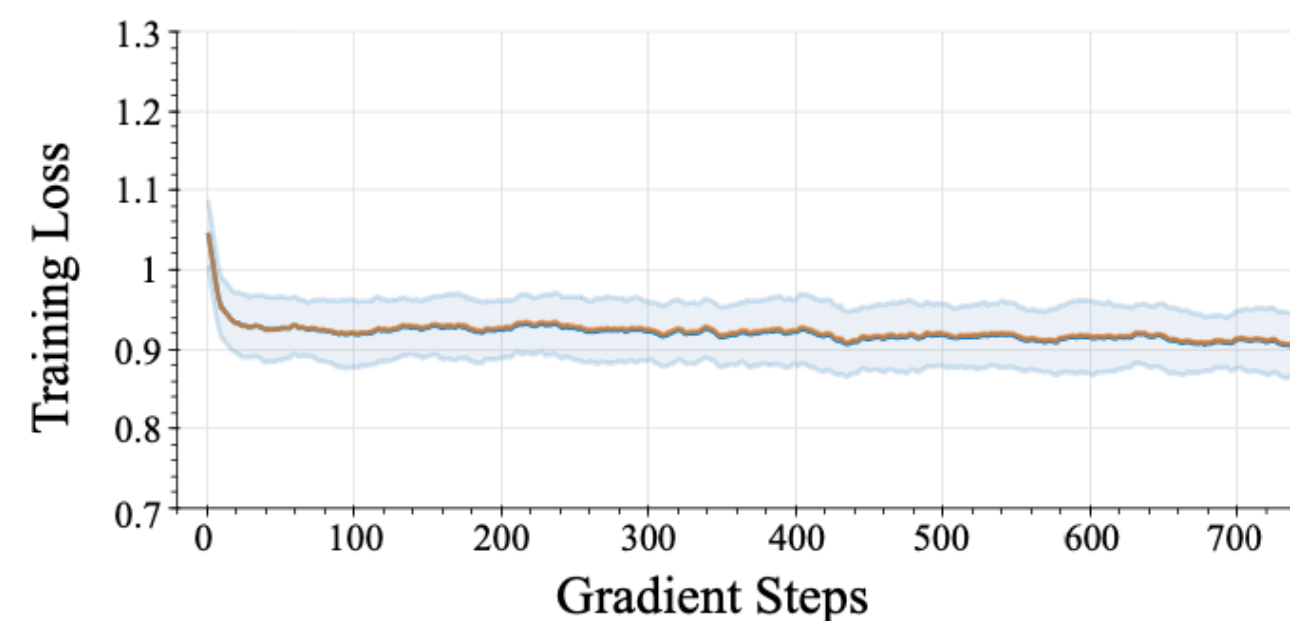
Это стоило проверить, так как фильтрация делала градиент приближённым.



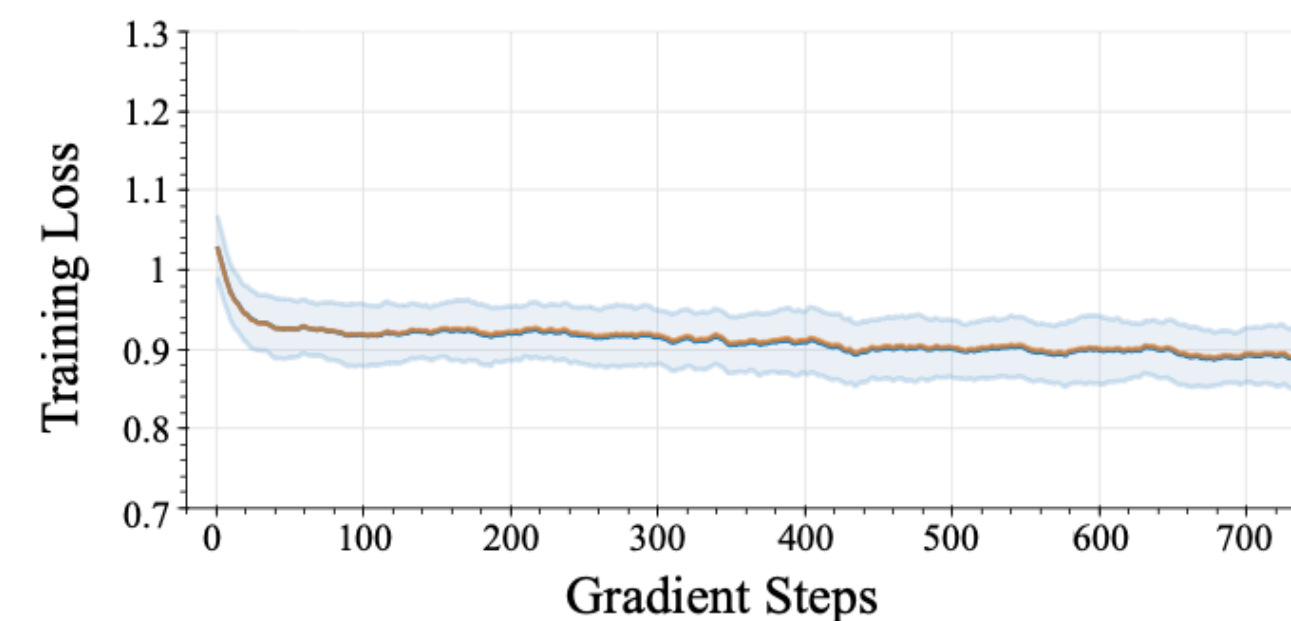
(a) Gemma 2 2B



(b) Phi 3.5 Mini



(c) Qwen 2.5 7B

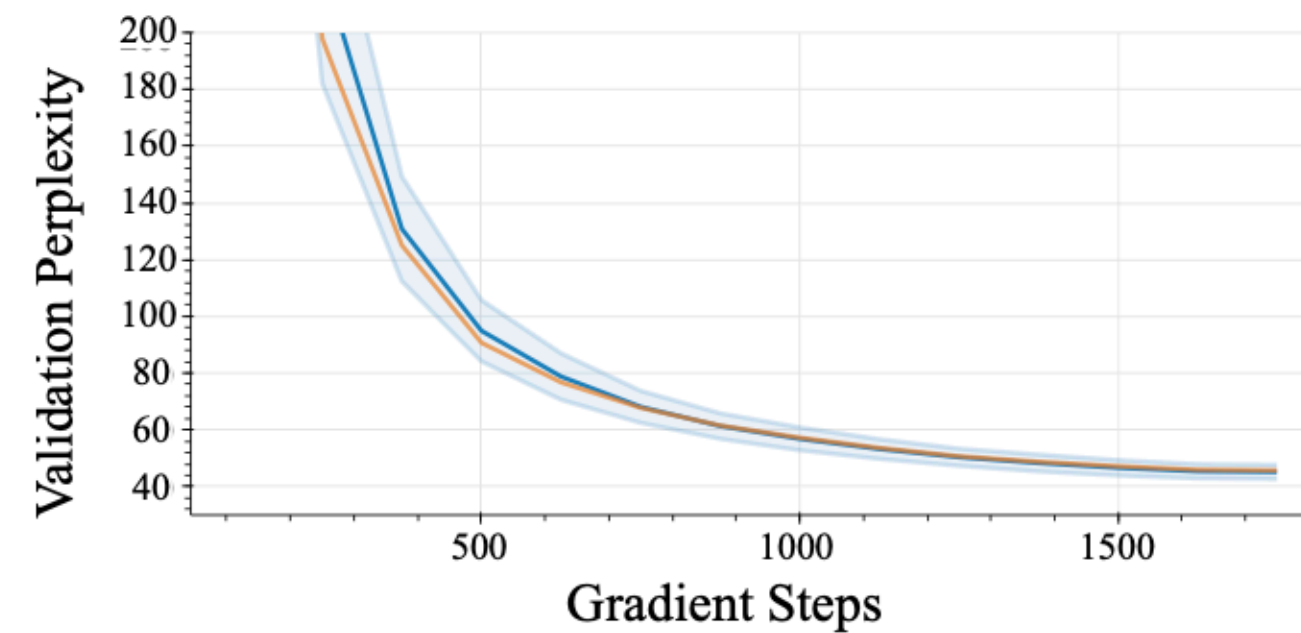


(d) Mistral Nemo

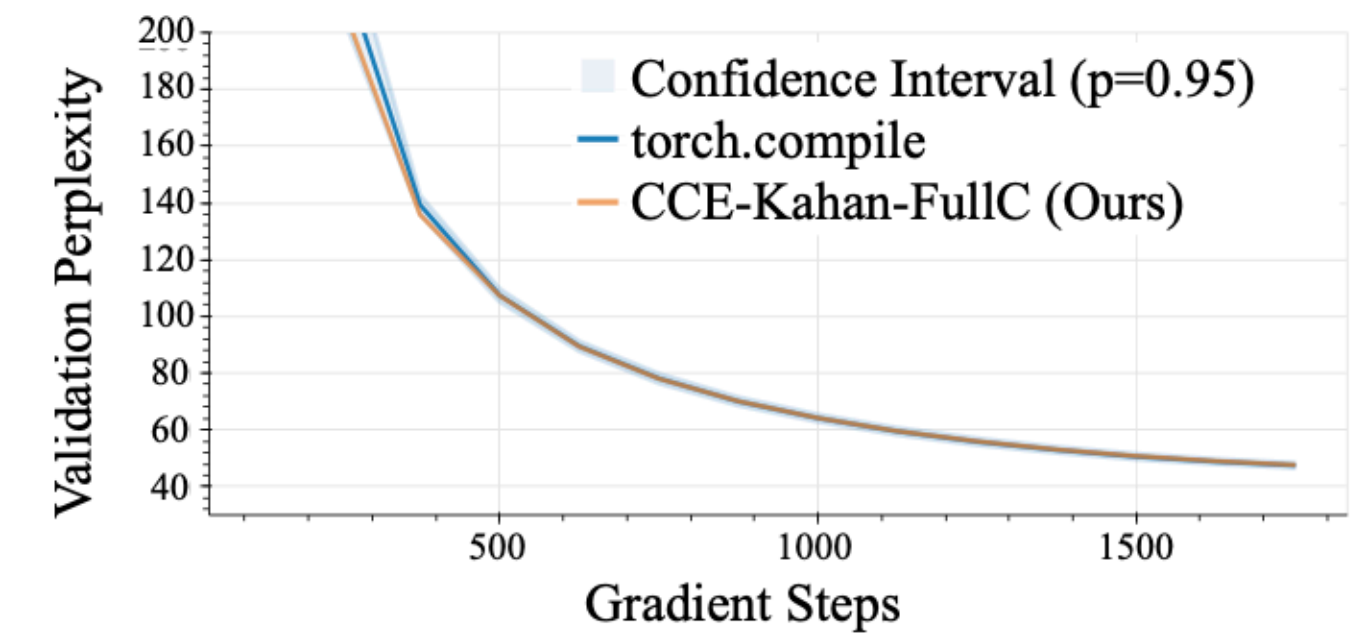
Результаты

Pretrain оказался чувствительнее, нашли две проблемы:

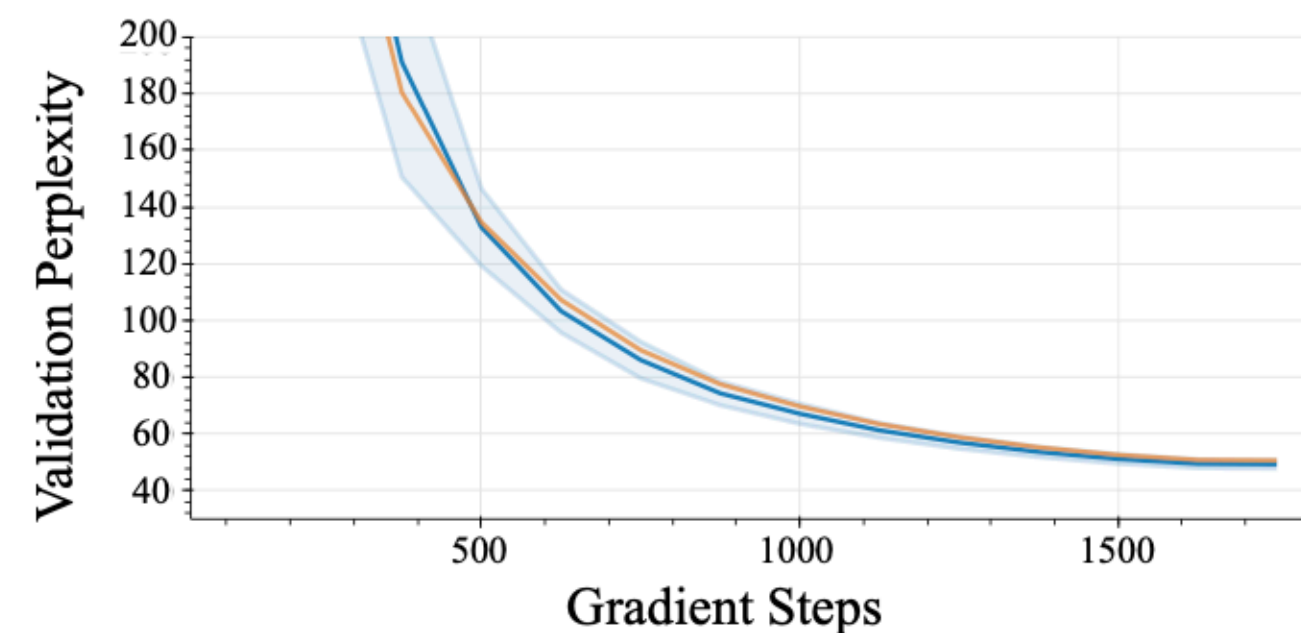
- фильтрация по градиенту C не даёт градиента редким словам, которые почти не встречаются в данных. Лечится вариантом FullC: для градиента C фильтр отключают;
- атомарные суммы в HBM идут в bf16 и теряют точность - лечится поправкой в подходе суммирования Кахана.



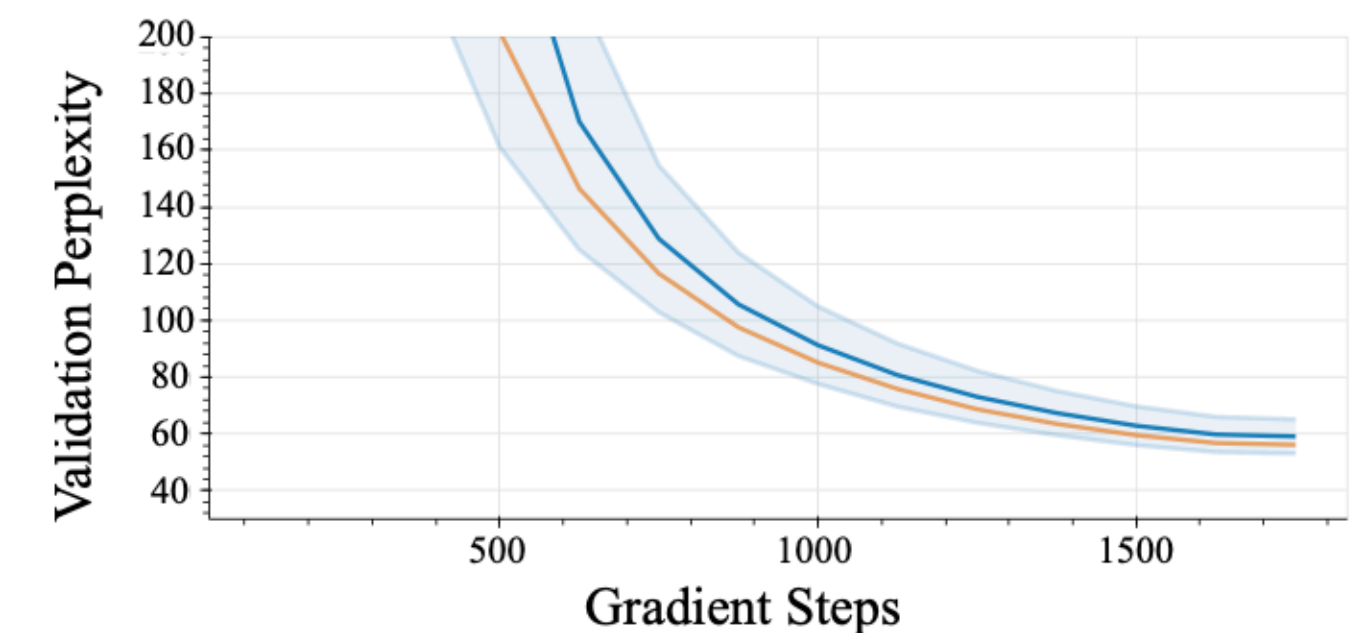
(a) Gemma 2 2B



(b) Phi 3.5 Mini



(c) Qwen 2.5 7B



(d) Mistral Nemo